

Comprehensive Self-Evolution Evaluation Guide

Overview

This document provides complete specifications, datasets, and run commands for evaluating SelfEvolve’s self-evolution capabilities across 11 long-horizon tasks spanning 9 professional domains. The evaluation is organized into three categories based on the type of self-evolution capability being tested:

- 1. Codebase Integration Tasks - Test integration with existing unknown codebases
 - 2. Cross-Session Composition Tasks - Test capability accumulation and reuse across sessions
 - 3. Interface Evolution Tasks - Test adaptive interface design with version detection
-

Task Catalog

Table 1: All Self-Evolution Tasks Overview

ID	Task Name	Domain	Type	Codebase/Dataset Size	Key Challenge
1	Salary Analyzer*	HR/Employment	Integration	597 lines, 12 classes	Multi-file codebase analysis
2	Patient Risk Analyzer	Healthcare	Integration	450 lines, 5 classes	Nested object relationships
3	GPA Calculator	Education	Integration	520 lines, 5 classes	Weighted metric calculation
4	Inventory Replenishment	E-commerce	Integration	680 lines, 6 classes	Time-series sales prediction
5	Matrix Eigenvalue*	Scientific Computing	Composition	2 sessions	Cross-session dependency
6	Portfolio Risk	Finance	Composition	2 sessions	Statistical composition
7	IoT Sensor Pipeline	IoT	Composition	2 sessions	Protocol parsing + aggregation
8	Movie API*	Entertainment	Evolution	27,279 movie records	3-version compatibility

9	Book Recommender	Library	Evolution	100 book records	Stack inspection versioning
10	Performance Tracker	HR	Evolution	50 review records	Role-based data filtering
11	Friend Suggester	Social Network	Evolution	100 user graph	Progressive enhancement

* Indicates tasks already evaluated in the paper

Table 2: Task Characteristics by Category

Category	Tasks	Avg Codebase Size	Avg Classes	Integration Complexity
Codebase Integration	4	587 lines	9.5 classes	High (multi-class navigation)
Cross-Session Composition	3	2 sessions each	N/A	Medium (context retrieval)
Interface Evolution	4	7,107 records avg	N/A	Medium (version detection)

Table 3: Domain Coverage

Domain	Task Count	Represents
Healthcare	1	Medical decision support systems
Education	1	Academic information systems
E-commerce	1	Supply chain management
Finance	1	Investment portfolio management
IoT	1	Smart device ecosystems
Library Management	1	Information retrieval systems
Human Resources	2	Employee management + performance systems
Social Networks	1	Graph-based recommendation systems

Scientific Computing	1	Numerical computation libraries
Total Domains	9	Enterprise + Consumer Software

Part 1: Codebase Integration Tasks

These tasks test the system's ability to analyze existing codebases and generate functions that correctly interface with discovered APIs and data structures.

Task 1: Salary Analyzer (Original - From Paper)

Domain: Human Resources / Employee Management

Existing Codebase:

- Location: test_context/ directory
- Size: 597 lines across 4 Python files
- Structure: 59 functions, 12 classes
- Key Classes:
 - Employee - Employee personal information
 - Manager - Extends Employee with team management
 - Department - Organizational units
 - EmployeeDatabase - Main database interface
 - SalaryCalculator - Salary computation utilities
 - Various helper classes for benefits, payroll, etc.

Task Description: Generate a function that analyzes employee salary data from an unfamiliar codebase, calculating average salaries across different employee types, departments, or tenure levels.

Run Command:

```
cd
```

```
/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks
```

```
python Core/main.py --clean-all --analyze test_context --request "Create a function called salary_analyzer that calculates average salary from employee data"
```

Expected Outcome:

- System analyzes 4-file codebase
 - Discovers database access methods
 - Generates function compatible with existing data structures
 - Success in 2 iterations with TDD (168s), failure without TDD (179s, 6 iterations)
-

Task 2: Medical Patient Risk Analyzer

Domain: Healthcare

Existing Codebase:

- Location: test_context_extended/patient_records.py
- Size: 151 lines
- Structure: 5 classes with sample data
- Key Classes:

class Hospital:

- patients: Dict[str, Patient]
- records: List[MedicalRecord]
- get_patient(patient_id) -> Patient
- get_records_by_patient(patient_id) -> List[MedicalRecord]
- get_recent_records(patient_id, days) -> List[MedicalRecord]

class Patient:

- id, name, age, blood_type
- conditions: List[Condition]
- get_total_conditions() -> int
- get_average_severity() -> float

class Condition:

- name, severity (1-10), diagnosed_date

class MedicalRecord:

- patient_id, visit_date, vitals (dict), medications
- has_abnormal_vitals() -> bool
-

Sample Data:

- 3 patients (high risk, medium risk, low risk)
- Multiple conditions per patient
- Recent medical records with vital signs

Task Description: Generate a patient_risk_score function that calculates patient risk (0-100) using weighted factors:

- Age: 30% weight
- Number of conditions: 25% weight
- Average condition severity: 25% weight
- Abnormal vitals in recent records: 20% weight

Run Command:

cd

/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks

python Core/main.py --clean-all --analyze test_context_extended --request "Create a function called patient_risk_score that analyzes a patient's medical history from the Hospital system and calculates their risk score (0-100) based on: age (weight 0.3), number of conditions (weight 0.25), average condition severity (weight 0.25), and abnormal vitals in recent records (weight 0.2). The function should take a Hospital object and patient_id as parameters."

Expected Outcome:

- System discovers Hospital.get_patient() and Patient.get_average_severity()
 - Navigates nested relationships (Hospital → Patient → Conditions)
 - Correctly weighs multiple risk factors
 - Returns valid risk score between 0-100
-

Task 3: Student GPA Calculator

Domain: Education

Existing Codebase:

- Location: test_context_extended/enrollment_system.py
- Size: 163 lines
- Structure: 5 classes with academic rules
- Key Classes:

class University:

- students: Dict[str, Student]
- courses: Dict[str, Course]
- enrollments: List[Enrollment]
- get_student_enrollments(student_id) -> List[Enrollment]
- get_completed_credits(student_id) -> int

class Course:

- code, name, credits, difficulty (100-400)
- is_advanced() -> bool

class Enrollment:

- student_id, course_code, semester, grade
- get_grade_points() -> float # Converts A/B/C to 4.0/3.0/2.0

class Student:

- id, name, major, admission_year

Sample Data:

- 3 students with varying performance
- 7 courses across difficulty levels (100-400)
- Multiple enrollments with letter grades

Task Description: Generate a calculate_cumulative_gpa function that computes weighted GPA:

- 400-level courses: 1.2x multiplier

- 300-level courses: 1.1x multiplier
- 100-200 level: 1.0x multiplier
- Weight by course credits

Run Command:

cd

/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks

python Core/main.py --clean-all --analyze test_context_extended --request "Create a function called calculate_cumulative_gpa that computes a student's cumulative GPA from the University system. Apply difficulty weighting: 400-level courses weighted 1.2x, 300-level weighted 1.1x, others 1.0x. Use course credits and enrollment grades. The function should take a University object and student_id as parameters and return the weighted GPA (0.0-4.0 scale)."

Expected Outcome:

- Discovers Course.difficulty and Enrollment.get_grade_points()
 - Implements difficulty-based weighting correctly
 - Handles credit-weighted GPA calculation
 - Returns value in valid 0.0-4.0 range
-

Task 4: E-Commerce Inventory Replenishment

Domain: Retail / Supply Chain

Existing Codebase:

- Location: test_context_extended/inventory_system.py
- Size: 187 lines
- Structure: 6 classes with sales tracking
- Key Classes:

class Warehouse:

- products: Dict[str, Product]
- stock_levels: List[StockLevel]
- sales_history: List[SalesTransaction]
- get_current_stock(sku, warehouse_id) -> int
- get_sales_last_n_days(sku, days) -> List[SalesTransaction]
- calculate_daily_sales_rate(sku, days) -> float
- get_low_stock_products() -> List[Tuple[Product, int]]

class Product:

- sku, name, category, reorder_point, lead_time_days

class StockLevel:

- sku, warehouse_id, quantity, last_updated
- is_low_stock(reorder_point) -> bool

class SalesTransaction:

- transaction_id, sku, quantity_sold, date, warehouse_id

Sample Data:

- 6 products with varying reorder points and lead times
- 30 days of sales history (120+ transactions)
- Current stock levels (some below reorder points)
- Varying sales velocities (1-8 units/day)

Task Description: Generate a generate_reorder_list function that:

1. Identifies products below reorder point
2. Calculates sales velocity from last 30 days
3. Projects days until stockout
4. Recommends order quantity considering lead time

Run Command:

cd

/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks

python Core/main.py --clean-all --analyze test_context_extended --request "Create a function called generate_reorder_list that identifies products needing reorder from the Warehouse system. Calculate sales velocity from last 30 days, project stock depletion date, and recommend reorder quantity considering lead time. Return a list of dictionaries with {sku, current_stock, daily_sales_rate, days_until_stockout, recommended_order_qty}. The function should take a Warehouse object as parameter."

Expected Outcome:

- Uses Warehouse.get_low_stock_products() and calculate_daily_sales_rate()
- Correctly projects stockout timeline
- Accounts for lead_time_days in recommendations
- Returns structured reorder list

Part 2: Cross-Session Composition Tasks

These tasks test the system's ability to preserve functions in Terminal Context and compose new capabilities atop previously generated code across different sessions.

Task 5: Matrix Eigenvalue (Original - From Paper)

Domain: Scientific Computing / Linear Algebra

Session Structure:

- Session 1: Generate matrix_operations() with caching

- Session 2: Generate `advanced_matrix_ops()` using Session 1 function

Task Description: Session 1 creates basic matrix operations. Session 2 retrieves this from Terminal Context and builds eigenvalue calculations atop it, demonstrating the system's ability to compose its own previously generated capabilities.

Run Commands:

Session 1:

```
cd
```

```
/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks
```

```
python Core/main.py --clean-all --request "Create a function called matrix_operations that performs basic matrix multiplication and stores results in a global variable computation_cache."
```

Session 2:

```
python Core/main.py --context-memory
```

When prompted, select the session containing `matrix_operations` (usually option 1)

Then enter the request when prompted:

Request for Session 2:

Load the previous session and create a function called `advanced_matrix_ops` that uses the existing `matrix_operations` function and `computation_cache` to perform eigenvalue calculations.

Expected Outcome:

- Session 1: Generates `matrix_operations` successfully
- Session 2: Retrieves function from context and composes eigenvalue calculator
- Success in 1 iteration with TDD (125s), failure without TDD (169s, 6 iterations)

Task 6: Financial Portfolio Risk Calculator

Domain: Finance / Investment Management

Session Structure:

- Session 1: Generate `calculate_stock_volatility()` for statistical analysis
- Session 2: Generate `portfolio_risk_assessment()` using volatility function

Session 1 Specification:

Goal: Create a function that calculates 30-day stock price volatility

Algorithm:

1. Calculate daily returns: $\text{return}[i] = (\text{price}[i] - \text{price}[i-1]) / \text{price}[i-1]$
2. Calculate mean return: $\text{mean} = \text{sum}(\text{returns}) / \text{len}(\text{returns})$
3. Calculate variance: $\text{var} = \text{sum}((r - \text{mean})^2 \text{ for } r \text{ in returns}) / \text{len}(\text{returns})$
4. Return standard deviation as percentage: $\text{sqrt}(\text{variance}) \times 100$

Edge Cases:

- Empty price list → return 0.0
- Single price → return 0.0
- Handle negative prices gracefully

Session 2 Specification:

Goal: Create portfolio-level risk using individual stock volatilities

Input Format:

```
portfolio = {
  'AAPL': {'prices': [150.0, 152.0, 148.0, 155.0], 'allocation': 0.4},
  'GOOGL': {'prices': [2800.0, 2820.0, 2790.0, 2850.0], 'allocation': 0.6}
}
```

Algorithm:

1. For each stock:
 - Extract price history
 - Call `calculate_stock_volatility(prices)`
 - Multiply volatility by allocation percentage
2. Sum all weighted volatilities
3. Return overall portfolio risk

Run Commands:

Session 1:

`cd`

`/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks`

`python Core/main.py --clean-all --request "Create a function called calculate_stock_volatility that takes a list of daily stock prices and calculates the 30-day volatility (standard deviation of daily returns as percentage). Calculate daily returns as $(\text{price}[i] - \text{price}[i-1]) / \text{price}[i-1]$, then return standard deviation times 100."`

Session 2:

`python Core/main.py --context-memory`

`# Select session with calculate_stock_volatility`

Request for Session 2:

Load the previous session and create a function called `portfolio_risk_assessment` that uses the existing `calculate_stock_volatility` function. The new function should take a dictionary where each key is a stock symbol and value is a dict with 'prices' (list of floats) and 'allocation' (float

0-1). Calculate volatility for each stock, weight by allocation, and return overall portfolio risk as weighted average.

Expected Outcome:

- Session 1: Volatility calculator with proper statistical formulas
 - Session 2: Successfully retrieves and composes weighted portfolio calculation
 - Demonstrates financial domain self-evolution
-

Task 7: IoT Sensor Data Pipeline

Domain: IoT / Smart Home Systems

Session Structure:

- Session 1: Generate `parse_sensor_reading()` for protocol parsing
- Session 2: Generate `aggregate_room_temperature()` using parser

Session 1 Specification:

Protocol Format: 'T:25.5|H:60|TS:1234567890'

Field Definitions:

- T = Temperature in Celsius (float)
- H = Humidity percentage (float)
- TS = Unix timestamp (integer)

Parsing Requirements:

Input: 'T:25.5|H:60|TS:1696800000'

Output: {'temperature': 25.5, 'humidity': 60.0, 'timestamp': 1696800000}

Malformed input: 'T:invalid|H:60|TS:abc'

Output: {'temperature': None, 'humidity': 60.0, 'timestamp': None}

Session 2 Specification:

Goal: Aggregate temperature readings with time filtering

Algorithm:

1. Parse all raw sensor readings using `parse_sensor_reading()`
2. Find most recent timestamp
3. Filter readings within last 60 minutes (3600 seconds)
4. Calculate average temperature from filtered readings
5. Return rounded to 2 decimal places

Edge Cases:

- No valid readings → return 0.0
- All readings older than 60 minutes → return 0.0

Run Commands:

Session 1:

```
cd
```

```
/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks
```

```
python Core/main.py --clean-all --request "Create a function called parse_sensor_reading that parses IoT sensor data format 'T:25.5|H:60|TS:1234567890' where T=temperature, H=humidity, TS=Unix timestamp. Return dictionary with keys: temperature, humidity, timestamp. Handle malformed data by returning None for invalid fields."
```

Session 2:

```
python Core/main.py --context-memory
```

```
# Select session with parse_sensor_reading
```

Request for Session 2:

Load the previous session and create a function called `aggregate_room_temperature` that uses the existing `parse_sensor_reading` function. Take a list of raw sensor readings, parse each one, filter readings from the last 60 minutes (3600 seconds from most recent timestamp), and return the average temperature rounded to 2 decimal places.

Expected Outcome:

- Session 1: Robust protocol parser with error handling
 - Session 2: Time-aware aggregation using retrieved parser
 - Demonstrates IoT domain self-evolution
-

Part 3: Interface Evolution Tasks

These tasks test the system's ability to detect caller context and adaptively return different data formats, demonstrating runtime interface evolution.

Task 8: Movie API (Original - From Paper)

Domain: Entertainment / Data Services

Dataset:

- Location: `movielens_dataset/movie.csv`
- Size: 27,279 movie records
- Source: MovieLens 20M Dataset
- Fields: `movieId`, `title`, `genres` (pipe-separated)

Sample Records:

movieId,title,genres

1,"Toy Story (1995)","Adventure|Animation|Children|Comedy|Fantasy"

2,"Jumanji (1995)","Adventure|Children|Fantasy"

Task Description: Generate a movie_api function that inspects the Python call stack to detect caller version and returns appropriate format:

Version Detection Logic:

```
import inspect
caller_name = inspect.stack()[1].function

if caller_name.endswith('_v1'):
    return ['Toy Story (1995)', 'Jumanji (1995)', ...] # Simple list
elif caller_name.endswith('_v2'):
    return {'titles': [...], 'count': 2, 'source': 'movie.csv'} # Metadata dict
elif caller_name.endswith('_v3'):
    return {'@context': 'http://schema.org', '@type': 'MovieCollection', ...} # JSON-LD
```

Run Command:

cd

/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks

python Core/main.py --clean-all --request "Create a function called movie_api that reads movielens_dataset/movie.csv and returns movie data in three different formats based on caller context. Inspect call stack to detect version: if caller ends with '_v1' return simple list of titles, if '_v2' return dict with titles and metadata, if '_v3' return JSON-LD format. Use inspect module."

Expected Outcome:

- Implements call stack inspection correctly
 - Returns three different formats
 - Maintains backward compatibility across versions
-

Task 9: Library Book Recommender

Domain: Library Management / Information Systems

Dataset:

- Location: test_context_extended/books.csv
- Size: 100 book records
- Fields: isbn, title, author, genre, year, rating
- Format: CSV with header

Sample Records:

isbn,title,author,genre,year,rating

978-6714714797,Book Title 1,Gabriel Garcia Marquez,Non-Fiction,2019,3.1

978-2300828189,Book Title 2,Agatha Christie,Thriller,2013,3.5

Genre Distribution:

- 10 genres: Fiction, Non-Fiction, Science Fiction, Mystery, Romance, Thriller, Biography, History, Self-Help, Fantasy
- 15 authors including J.K. Rowling, Stephen King, Agatha Christie, etc.
- Ratings: 3.0-5.0 scale
- Publication years: 1990-2024

Task Description: Generate a book_recommender function that filters by genre and returns data in three formats based on caller version detection.

Version Formats:

v1 (Legacy):

Caller: legacy_report_v1()

Returns: ["Book Title 1", "Book Title 5", "Book Title 12", ...]

v2 (Metadata):

Caller: analytics_v2()

Returns: {

"titles": ["Book Title 1", "Book Title 5", ...],

"count": 15,

"avg_rating": 4.2,

"genres": ["Fiction"]

}

v3 (JSON-LD):

Caller: api_export_v3()

Returns: {

"@context": "http://schema.org",

"@type": "BookCollection",

"name": "Fiction Books",

"numberOfItems": 15,

"hasPart": [

{"@type": "Book", "name": "Book Title 1", "author": "J.K. Rowling"},

...

]

}

Run Command:

cd

/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks

python Core/main.py --clean-all --request "Create a function called book_recommender that reads test_context_extended/books.csv and recommends books based on genre parameter.

The function must inspect the Python call stack to detect which version the caller expects: if caller function name ends with '_v1' return list of titles, if ends with '_v2' return dict with titles/count/avg_rating/genres, if ends with '_v3' return JSON-LD format with

@context=http://schema.org and @type=BookCollection. Use the inspect module to check stack frames."

Expected Outcome:

- Correctly reads and parses CSV
 - Implements stack inspection for version detection
 - Returns all three formats without errors
 - Properly filters by genre parameter
-

Task 10: HR Performance Review Tracker

Domain: Human Resources / Performance Management

Dataset:

- Location: test_context_extended/performance_reviews.csv
- Size: 50 employee review records
- Fields: employee_id, employee_name, department, position, review_period, technical_score, communication_score, teamwork_score, leadership_score, overall_rating, goals_status, feedback

Sample Records:

employee_id,employee_name,department,position,review_period,technical_score,communication_score,teamwork_score,leadership_score,overall_rating,goals_status,feedback

E001,Employee 1,Operations,Manager,Q1 2024,3,4,5,4,4.0,Met,Performance feedback for Employee 1

E002,Employee 2,Customer Support,Senior,Q3 2024,4,3,4,3,3.5,Exceeded,Performance feedback for Employee 2

Data Distribution:

- 7 departments: Engineering, Sales, Marketing, HR, Finance, Operations, Customer Support
- 5 position levels: Junior, Mid-level, Senior, Lead, Manager
- Review periods: Q1-Q4 2024
- Scores: 1-5 scale (0 for non-applicable leadership scores)
- Goals status: Exceeded, Met, Partially Met, Not Met

Task Description: Generate a performance_report function that detects caller role and returns appropriate detail level:

Manager View:

```
# Caller: team_review_manager()
# Input: employee_id="E001"
# Returns: {
#   "employee_id": "E001",
#   "overall_rating": 4.0,
#   "goals_status": "Met"
# }
```

HR View:

```
# Caller: full_review_hr()
# Input: employee_id="E001"
# Returns: {
#   "employee_id": "E001",
#   "employee_name": "Employee 1",
#   "department": "Operations",
#   "technical_score": 3,
#   "communication_score": 4,
#   "teamwork_score": 5,
#   "leadership_score": 4,
#   "overall_rating": 4.0,
#   "goals_status": "Met",
#   "feedback": "Performance feedback for Employee 1"
# }
```

Executive View:

```
# Caller: department_summary_executive()
# Input: employee_id=None
# Returns: {
#   "Engineering": {"avg_rating": 3.8, "count": 7, "exceeded_goals": 3},
#   "Sales": {"avg_rating": 4.1, "count": 6, "exceeded_goals": 2},
#   ...
# }
```

Run Command:

```
cd
/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks
```

python Core/main.py --clean-all --request "Create a function called performance_report that reads test_context_extended/performance_reviews.csv and generates different reports based on caller role. Inspect the calling function name: if contains '_manager' return summary scores only (employee_id, overall_rating, goals_status), if contains '_hr' return all details including feedback comments, if contains '_executive' and employee_id is None then return aggregated stats by department (average ratings, count). Use inspect module to check caller function name."

Expected Outcome:

- Correctly detects three different caller roles
 - Returns appropriate detail level for each role
 - Executive view properly aggregates by department
 - Handles None employee_id for executive aggregation
-

Task 11: Social Network Friend Suggester

Domain: Social Networks / Recommendation Systems

Dataset:

- Location: test_context_extended/social_graph.json
- Size: 100 user profiles
- Format: JSON with nested user objects

Data Structure:

```
{
  "users": [
    {
      "user_id": "U001",
      "username": "user1",
      "friends": ["U052", "U074", "U054", ...],
      "interests": ["Travel", "Reading", "Fashion"],
      "activity": {
        "level": "medium",
        "posts_per_week": 6,
        "likes_per_week": 91,
        "comments_per_week": 12
      },
      "member_since": "2019-12-14"
    },
    ...
  ],
  "total_users": 100
}
```


Dataset Statistics:

- Average friends per user: 9.8
- Average interests per user: 4.2
- 14 interest categories: Photography, Hiking, Reading, Cooking, Gaming, Music, Travel, Sports, Art, Technology, Fitness, Movies, Fashion, Gardening
- Activity levels: low (29%), medium (27%), high (24%), very_high (20%)

Task Description: Generate a friend_suggestions function with progressive enhancement based on detected mode.

Mode 1: Basic

```
# Environment: RECOMMENDATION_MODE='basic'
# Algorithm: Find users with mutual friends
# Scoring: score = number_of_mutual_friends
# Returns: Top 5 users sorted by score
```

Mode 2: Enhanced

```
# Environment: RECOMMENDATION_MODE='enhanced'
# Algorithm: Mutual friends + common interests
# Scoring: score = (mutual_friends × 2.0) + (common_interests × 1.5)
# Returns: Top 5 users sorted by score
```

Mode 3: Advanced

```
# Environment: RECOMMENDATION_MODE='advanced'
# Algorithm: Mutual friends + interests + activity similarity
# Scoring: score = (mutual_friends × 2.0) + (common_interests × 1.5) + (activity_match × 1.0)
# Activity match: 1.0 if same level, 0.5 if adjacent levels, 0.0 otherwise
# Returns: Top 5 users sorted by score with detailed reasoning
```

Run Command:

```
cd
/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks
```

```
python Core/main.py --clean-all --request "Create a function called friend_suggestions that reads test_context_extended/social_graph.json and recommends friends with progressive features based on RECOMMENDATION_MODE environment variable. In 'basic' mode: find users with mutual friends, return top 5 with score = number of mutual friends. In 'enhanced' mode: add common interests to score (mutual_friends * 2 + common_interests * 1.5). In 'advanced' mode: add activity similarity (mutual_friends * 2 + interests * 1.5 + activity_match * 1.0). Check os.environ.get('RECOMMENDATION_MODE', 'basic') to determine mode."
```

Expected Outcome:

- Reads and parses JSON graph structure
 - Detects mode from environment variable
 - Implements three progressively complex algorithms
 - Returns valid recommendations with scores
-

Complete Dataset Specifications

Table 4: Dataset Characteristics

Dataset	Type	Size	Records	Fields	Domain
test_context/	Python codebase	597 lines	12 classes	59 functions	Employee management
patient_records.py	Python codebase	151 lines	5 classes	3 patients	Medical records
enrollment_system.py	Python codebase	163 lines	5 classes	3 students, 7 courses	University enrollment
inventory_system.py	Python codebase	187 lines	6 classes	6 products, 120+ sales	Warehouse management
movielens_dataset/movie.csv	CSV	27,279 rows	Movies	3 (id, title, genres)	Entertainment catalog
books.csv	CSV	100 rows	Books	6 (isbn, title, author, genre, year, rating)	Library catalog
performance_reviews.csv	CSV	50 rows	Reviews	11 (scores, feedback, department)	Performance management
social_graph.json	JSON	100 users	User profiles	6 (id, friends, interests, activity, etc.)	Social network

Table 5: Codebase Complexity Metrics

Task	Files	Lines	Classes	Methods	Complexity
Salary Analyzer	4	597	12	59	High (multi-file)
Patient Risk	1	151	5	15	Medium (nested objects)
GPA Calculator	1	163	5	18	Medium (weighted calc)
Inventory System	1	187	6	22	High (temporal analysis)

Execution Instructions

General Workflow

All tasks follow this pattern:

1. Navigate to framework directory:
cd
/home/aifahim/PycharmProjects/Self-Adapting-AI-Agent/framework_our_approach/ablation_study_humaneval_and_long_horizon_tasks
2. Run task with appropriate flags:
 - --clean-all: Clear previous functions and tests
 - --analyze <path>: Provide codebase directory for integration tasks
 - --request "<description>": Specify function requirements
 - --context-memory: Enable cross-session composition
3. Monitor execution:
 - Watch iteration count (expect 1-3 with TDD)
 - Check function_generation.log for diagnostic feedback
 - Verify in functions.py when complete
4. Verify success:
 - Final adjudication shows PASS
 - Generated function in functions.py
 - Tool descriptor in Tool_Descriptor_Gen/tools.json

Task-Specific Execution Order

Integration Tasks (Run directly):

- Task 1: Salary Analyzer

- Task 2: Patient Risk Analyzer
- Task 3: GPA Calculator
- Task 4: Inventory Replenishment

Composition Tasks (Require 2 sessions):

- Task 5: Matrix Eigenvalue (Session 1 → Session 2)
- Task 6: Portfolio Risk (Session 1 → Session 2)
- Task 7: IoT Sensor Pipeline (Session 1 → Session 2)

Evolution Tasks (Run directly):

- Task 8: Movie API
 - Task 9: Book Recommender
 - Task 10: Performance Tracker
 - Task 11: Friend Suggester
-

Evaluation Metrics

For each task, measure:

1. Success/Failure: Did the function pass final adjudication?
2. Iteration Count: How many refinement cycles (expect 1-3 with TDD)?
3. Execution Time: Total time from request to completion
4. Code Quality: Does generated code correctly use discovered APIs?

Aggregated Metrics:

- Pass@1 Rate: Percentage of tasks succeeding
 - Average Iterations: Mean iteration count for successful tasks
 - Domain Coverage: Number of domains with successful tasks
-

Expected Research Outcomes

This comprehensive evaluation enables:

1. Quantitative Validation:
 - HumanEval Pro: 68.4% Pass@1 on composition tasks
 - Self-evolution tasks: Success rates across 11 diverse scenarios
2. Qualitative Validation:
 - 9 professional domains demonstrate broad applicability
 - Three evolution patterns (integration, composition, interface) comprehensively tested
3. Statistical Analysis:
 - Multiple runs per task enable Wilcoxon testing
 - Domain-stratified analysis shows generalization
4. Architectural Insights:
 - TDD impact across diverse domains
 - Terminal Context effectiveness for composition
 - Dual adjudication robustness across complexity levels

This expanded evaluation provides rigorous evidence that SelfEvolve achieves practical self-evolution across realistic software engineering scenarios.